Dependent Type Theory, an Introduction

Claudio Bastiani-Fonck

1. Introduction

Proof assistants such as Lean are rapidly gaining popularity. It is possible that in the future these systems might play a very nontrivial role in how mathematics is done. The following is a conceptual introduction to the basic concepts in dependent type theory. The purpose is to give some broad intuition for what these systems are actually doing under the hood in order to verify that a formal proof is correct. Throughout I will use Lean-style conventions: propositions live in Prop, existential quantification is the inductive proposition Exists, and propositional equality is Eq. I will suppress most universe levels, coercions, and similar elaboration details when they are not important.

2. Curry–Howard

The Curry–Howard correspondence states essentially that there is a correspondence:

$$\text{Mathematical Proofs} \qquad \longleftrightarrow \qquad \text{Computer Programs}$$

This is not very intuitive, so we will give an example.

Consider the statement that $f(x) = x^2$ is continuous at $x = 2$. If you've taken analysis or the 160s, you'll know that this is a statement about epsilons and deltas:

$$\forall \varepsilon \in \mathbb{R}_{>0} \qquad \exists \delta \in \mathbb{R}_{>0} \qquad \forall x \in \mathbb{R} \qquad |x-2| < \delta \implies |x^2 - 4| < \varepsilon \tag{1}$$

The ordinary mathematical proof is the following.

Fix $\varepsilon > 0$ and choose:

$$\delta := \min\left(1, \frac{\varepsilon}{5}\right)$$

Both 1 and $\varepsilon/5$ are positive, so $\delta > 0$. Suppose $|x-2| < \delta$. Since $\delta \leq 1$:

$$|x+2| = |(x-2)+4| \leq |x-2| + 4 < \delta + 4 \leq 5$$

Also $\delta \leq \varepsilon/5$. Therefore:

$$|x^2 - 4| = |x-2|\,|x+2| < 5\delta \leq \varepsilon$$

This proves (1).

Now we will see how the mathematical proof translates to a computer program, and moreover how the statement is translated into a type.

2.1. Types

When we do dependent type theory, the objects manipulated by the computer are called *terms*. Mathematically, a term is a formal expression built from symbols like $\lambda, \longrightarrow, \ldots$ according to precise rules, but generally they represent computational data such as numbers, functions, mathematical statements, etc. We will assume throughout that all terms under discussion are *well-typed*, meaning the typing rules assign them a type.

Every term has a *type* which is itself another term. We write

$$a : A$$

to express that a has type A. For instance $3 : \mathbb{N}$, $v : \mathbb{R}^n$, or $f : A \longrightarrow B$ (for a term which maps something of type A to type B). The upshot is that a mathematical proposition can be expressed as some term $P : \mathsf{Prop}$. A proof of that proposition is a term p of type P, written as $p : P$.

As a warning, it is a common mistake to think of a type as some kind of set, and read $a : A$ as somehow analogous to $a \in A$. This is the wrong intuition for the following reason:

The type of a term is *intrinsic*, every term has a *unique* type (up to definitional equality)

On the other hand set membership is an extrinsic property. A useful heuristic is that it makes little sense to speak of "the set of x," whereas it is perfectly natural to speak of "the type of x." Once a term has been fully specified, its type is determined by the formal rules of dependent type theory and can be computed by an algorithm in finite steps. In this sense, the type of something expresses *exactly* what kind of data it is, something the computer can keep track of. Moreover $a : A$ is not some logical statement that a has type A, it is a *typing judgment*: the typechecker has computed the type of a to be A.

For example, suppose n is an integer and we prove that it is positive. If `Nat` is defined as the type of natural numbers, then n itself does not suddenly have type `Nat`; we must construct from n and the proof that n is positive a new object of type `Nat`.

Some of these statements have asterisks in some type theories. But since we are mainly concerned with Lean I will ignore that.

2.2. The type of a proof

As above, we consider the proposition that $|x^2 - 4| < \epsilon$ as a type, and a term of this type is a proof that $|x^2 - 4| < \epsilon$. We express the implication that

$$|x - 2| < \delta \implies |x^2 - 4| < \epsilon \tag{2}$$

as a *function* which takes a proof of $|x - 2| < \delta$, and returns a proof that $|x^2 - 4| < \epsilon$. The type of such a function is simply:

$$|x - 2| < \delta \longrightarrow |x^2 - 4| < \epsilon$$

However this type only expresses this implication for some given x. In order to express that

$$\forall x \qquad |x - 2| < \delta \implies |x^2 - 4| < \epsilon \tag{3}$$

we need a general function that takes an x and returns a proof that Equation (3) is satisfied for that x. The type signature of such a function is written as:

$$\prod_{x:\mathbb{R}} |x - 2| < \delta \longrightarrow |x^2 - 4| < \varepsilon$$

The $\prod$ expresses that the type of the output depends on the specific input of the function. Lean normally writes this as $\forall x : \mathbb{R}, \ldots$; I use $\prod$ to emphasize the underlying dependent function type. Ordinary function types have a fixed codomain. Dependent function types break this restriction, and can be vaguely thought of as the type-theoretic version of $\forall$.

To prove the theorem, we need a function which takes epsilons and returns a proof that an appropriate delta exists and satisfies Equation (3). Under Curry–Howard, the type of such a function is:

$$\prod_{\varepsilon:\mathbb{R}_{>0}} \exists\, \delta : \mathbb{R}_{>0}, \prod_{x:\mathbb{R}} \big(|x - 2| < \delta \longrightarrow |x^2 - 4| < \varepsilon\big) \tag{4}$$

Here $\mathbb{R}_{>0}$ is being used as shorthand for the subtype of positive real numbers. Read (4) from left to right:

1. The $\prod_\varepsilon$ means the proof is a function that receives a positive real number ε as input.

2. The $\exists\,\delta$ is Lean's inductive proposition Exists. To prove it, we give a witness δ together with a proof of the remaining proposition. Lean's anonymous constructor notation for this is $\langle \delta, h \rangle$. Because Exists lives in Prop, this is an existential proof rather than a data-carrying dependent pair that can be freely extracted into Type.

3. The final $\prod_x$ is part of that proof. It is a function that takes any real number x, assumes $|x - 2| < \delta$, and returns a proof of the required estimate $|x^2 - 4| < \varepsilon$.

Proving the continuity statement is the same as giving a term of this type.

2.3. Giving a term

In the notation of λ-calculus, the proof has the following term-shaped skeleton:

$$\begin{aligned} \mathsf{sqContAt2} := \lambda\varepsilon.\, \textbf{let}\ \delta := \min(1, \varepsilon/5)\ \textbf{in} \\ \langle \delta, \lambda x.\, \lambda h_x.\, h_{\text{estimate}} \rangle \end{aligned} \tag{5}$$

where, suppressing the subtype constructor proving that $\delta > 0$:

$$h_x : |x - 2| < \delta, \qquad h_{\text{estimate}} : |x^2 - 4| < \varepsilon$$

Thus the proof is a function from tolerances ε to proofs that a suitable tolerance δ exists, each built by giving a particular witness δ. To understand how this connects to the manual proof, consider how h_{estimate} is built. In a formal proof assistant, the ordinary chain of inequalities from our example box translates into a sequence of function applications. For instance, we might have a function of the form

$$x < y \longrightarrow y < z \longrightarrow x < z$$

which sends a proof that $x < y$ to a function which sends a proof of $y < z$ to a proof of $x < z$. This corresponds to transitivity of "less than". Each logical step, such as factoring a polynomial or applying this transitivity, corresponds to feeding some assumption into a pre-proven mathematical lemma. By composing these functions together, the program computes the final proof term h_{estimate}, and the kernel can easily check it has the desired type since it knows the type of each function we used.

3. Constructive mathematics

The Curry–Howard correspondence has some important practical implications with regard to other systems of formal logic. At its computational core, this sort of logical framework is constructive. Statements like IVT:

$$\text{for any continuous } f : [a, b] \to \mathbb{R} \text{ where } f(a) < 0 \text{ and } f(b) > 0, \text{ exists some } c \in [a, b] \text{ s.t. } f(c) = 0$$

are not constructively provable in this generality without introducing an additional classical principle such as the *law of excluded middle* (every statement is true or false). Intuitively, there is no general algorithm which computes such an exact root for *any* function f from these hypotheses alone.

This is prohibitive for formalizing ordinary classical mathematics. But it also makes the Curry–Howard correspondence very explicit. For instance, if I constructively prove that every number is even or odd, I have implicitly specified a computer program which determines whether any given number is even or odd. Moreover, in many proof assistants, I could automatically generate this computer code in a process called *extraction*.

Different proof assistants take different stances on this topic. For instance, Rocq by default does not have the law of excluded middle, however you can assert it as an axiom. Lean, on the other hand, being built for mathematics, has three standard mathematical axioms:

- propositional extensionality
- the axiom of choice

- quotient soundness

of which the axiom of choice is nonconstructive. These are enough to prove all the desired statements one wishes for in classical logic. Unfortunately, nonconstructive proofs do not in general carry executable computational content in the same way. In Lean, proofs in Prop are erased during compilation, so using classical reasoning inside a proof does not by itself prevent surrounding code from compiling; however, definitions which use classical choice to produce data are noncomputable.

4. The rules of dependent type theory

We saw in the above example that we required functions such that the type of the output depends on the specific input. Dependent type theory is the framework that makes this work.

A dependent type theory is organized around judgments made in a context Γ:

$$\Gamma \vdash A : \mathsf{Type} \qquad \text{and} \qquad \Gamma \vdash a : A$$

The first says that A is a well-formed type; the second says that a is a term of type A. A context is a list of typed variables, and later types may refer to earlier variables:

$$\Gamma = (A : \mathsf{Type},\ x : A,\ B : A \to \mathsf{Type},\ y : B(x))$$

4.1. Dependent function types

If $B(x)$ is a type for every $x : A$, then

$$\prod_{x:A} B(x)$$

is the type of functions f whose output $f(x)$ has type $B(x)$. The codomain may change with the input. The following are the formal rules that these types obey:

Formation, introduction, elimination, and computation for Π

$$\frac{\Gamma \vdash A : \mathsf{Type} \qquad \Gamma, x : A \vdash B(x) : \mathsf{Type}}{\Gamma \vdash \prod_{x:A} B(x) : \mathsf{Type}} \quad (\Pi\text{-form})$$

$$\frac{\Gamma, x : A \vdash b(x) : B(x)}{\Gamma \vdash \lambda x.\, b(x) : \prod_{x:A} B(x)} \quad (\Pi\text{-intro})$$

$$\frac{\Gamma \vdash f : \prod_{x:A} B(x) \qquad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B(a)} \quad (\Pi\text{-elim})$$

$$(\lambda x.\, b(x))(a) \equiv b(a) \qquad (\Pi\text{-computation, or } \beta\text{-rule})$$

If B does not depend on x, then $\prod_{x:A} B$ is just the ordinary function type $A \to B$. Logically, $\prod_{x:A} P(x)$ expresses $\forall x : A,\ P(x)$.

The symbol $\equiv$ denotes *definitional equality*: two expressions compute to the same normal form, such as $(\lambda x.b)(a) \equiv b[a/x]$. Effectively, there are many simple rules; for instance if f has type $A \longrightarrow B$ and x has type A, then $f(x)$ has type B. Two expressions are definitionally equal if they can be related to each other via these rules. The kernel checks definitional equality, and type checking uses definitional equality constantly. The proposition $a = b$ we will see later represents *propositional equality*, which requires a proof term.

5. Inductive types

Inductive types are the core mechanism by which mathematical objects are defined and data is reasoned about. An inductive type is specified by constructors, which are the mechanisms that are used to construct terms of that type.

5.1. Natural Numbers

For the natural numbers, the formation and constructor rules are:

$$\mathbb{N} : \mathsf{Type}, \qquad \mathsf{zero} : \mathbb{N}, \qquad \mathsf{succ} : \mathbb{N} \to \mathbb{N}$$

Meaning there is a natural number zero, and given any natural number we may consider its successor. Lean makes available a term of the following type, called the *recursor* or *eliminator*:

$$\mathsf{Nat.rec} : \prod_{P : \mathbb{N} \to \mathsf{Sort}\, u} P(\mathsf{zero}) \to \left(\prod_{n:\mathbb{N}} P(n) \to P(\mathsf{succ}(n)) \right) \to \prod_{n:\mathbb{N}} P(n) \tag{6}$$

Here P is the *motive*: a family of types indexed by n. The notation $\mathsf{Sort}\, u$ lets the same recursor cover both propositions and ordinary data types; $\mathsf{Sort}\, 0$ is Prop. Under Curry–Howard, if each $P(n)$ is a proposition, (6) is mathematical induction.

The computation rules say that induction computes on constructors:

$$\begin{aligned} \mathsf{Nat.rec}(P, z, s, \mathsf{zero}) &\equiv z, \\ \mathsf{Nat.rec}(P, z, s, \mathsf{succ}(n)) &\equiv s\big(n, \mathsf{Nat.rec}(P, z, s, n)\big) \end{aligned}$$

These are part of the meaning of $\mathbb{N}$, not additional theorems about it. The exact algorithm that the computer uses to generate the eliminator is something that I will skip. Intuitively, in order to map out of an inductive type, one needs to specify how the function unwraps the constructors, called *elimination*. For instance $f(x) = x^2$ can be defined by

$$f(0) \equiv 0 \qquad f(\mathsf{succ}(n)) \equiv f(n) + 2n + 1$$

assuming I have defined addition (coming up).

5.2. Equality

One of the most important instances of inductive types is the concept of *propositional equality*. In Lean-like notation

$$\begin{aligned} &\textbf{Inductive}\ \mathsf{Eq}\ \{A : \mathsf{Sort}\, u\}\ (a : A) : A \to \mathsf{Prop} := \\ &\quad \mid\ \mathsf{refl} : \mathsf{Eq}\ a\ a \end{aligned}$$

Lean writes $x = y$ as notation for $\mathsf{Eq}\ x\ y$. The constructor Eq.refl, usually written rfl, proves $a = a$. Because definitional equality is used during type checking, rfl can also prove an equality when the two sides reduce to definitionally equal terms. We say that $x : A$ and $y : A$ are *propositionally equal* if we can construct a term $p : x = y$. This is different from definitional equality since it can and must be proven. For example, propositional extensionality has type $(P \leftrightarrow Q) \to (P = Q)$, so this axiom constructs a term of this type. We can use this notion to prove some theorems.

5.3. Addition by zero

Define addition recursively on the first argument:

$$\mathsf{zero} + n \equiv n \qquad \mathsf{succ}(m) + n \equiv \mathsf{succ}(m + n)$$

I want to prove $n + \mathsf{zero} = n$ (these are not definitionally equal since we have put the zero on the other side). The statement $n + \mathsf{zero} = n$ has type:

$$\prod_{n:\mathbb{N}} (n + \mathsf{zero} = n)$$

It is proved using Nat.rec with motive:

$$P(n) := (n + \mathsf{zero} = n)$$

The base case is rfl. In the step, a proof $p : m + \mathsf{zero} = m$ is sent to the equality

$$\mathsf{succ}(m + \mathsf{zero}) = \mathsf{succ}(m)$$

obtained by applying succ to both sides of p. Thus recursion constructs data, while dependent recursion constructs proofs about that data.

5.4. Path induction

You might guess that the eliminator for equality is quite important. It is often called J or path induction; in Lean the relevant recursor is Eq.rec. In type-theoretic notation its essential type is:

$$\mathsf{Eq.rec}_{A,a} : \prod_{P:\left(\prod_{x:A}(a=x)\to \mathsf{Sort}\, u\right)} \left(P(a, \mathsf{rfl}) \longrightarrow \prod_{x:A} \prod_{p:a=x} P(x, p) \right)$$

It is what lets a proof assistant transport data and proofs across an equality. For instance, suppose I want to prove that if $x, y : A$ are propositionally equal, and T is a family of propositions indexed by A, then $T(x)$ implies $T(y)$. I desire to construct a term of type:

$$\prod_{x,y:A} (x = y) \longrightarrow T(x) \longrightarrow T(y) \tag{7}$$

I need to map out of an inductive type, so it's time to use the eliminator. Define the motive P as the function type that maps any endpoint z to $T(x) \to T(z)$:

$$P := \lambda(z : A)(q : x = z).(T(x) \to T(z))$$

By feeding this into $\mathsf{Eq.rec}_{A,x}$ I get a term of type:

$$\mathsf{Eq.rec}_{A,x}(P) : P(x, \mathsf{rfl}) \longrightarrow \prod_{z:A} \prod_{p:x=z} P(z, p)$$

Notice that $P(x, \mathsf{rfl}) \equiv T(x) \to T(x)$. The identity function is a term of this type, so feeding it into our expression gives a term of type:

$$\prod_{z:A} \prod_{p:x=z} T(x) \to T(z)$$

Plugging in y for z gives a term of type

$$\prod_{p:x=y} T(x) \to T(y)$$

and abstracting over x, y, and $p : x = y$ constructs a term of the desired type in (7). It is quite remarkable that the eliminator the kernel automatically generates when we define this specific inductive type gives us the machinery we need to perform this type of logical deduction. One would expect induction to be the most primitive form of logical argument.

6. More Inductives

You may have noticed that Eq was actually an indexed family of inductive propositions (despite my attempt to sweep it under the rug). I.e. for any type A and $a : A$, $\mathsf{Eq}\ a : A \to \mathsf{Prop}$ is a family of propositions indexed by the second endpoint. Lists are one of the simplest examples of an inductive type with a parameter, and vectors will make the indexing more visible.

6.1. Lists and Vectors

Fix a type $A : \mathsf{Type}$. The type of finite lists with entries in A is generated by:

$$\mathsf{List}(A) : \mathsf{Type} \qquad \mathsf{nil} : \mathsf{List}(A) \qquad \mathsf{cons} : A \to \mathsf{List}(A) \to \mathsf{List}(A)$$

Its dependent induction principle is:

$$\begin{aligned} \mathsf{List.rec}_A : & \prod_{P : \mathsf{List}(A) \to \mathsf{Sort}\, u} P(\mathsf{nil}) \\ & \to \left(\prod_{a : A} \prod_{\ell : \mathsf{List}(A)} P(\ell) \to P(\mathsf{cons}(a, \ell)) \right) \to \prod_{\ell : \mathsf{List}(A)} P(\ell) \end{aligned} \tag{8}$$

To define a function out of lists, specify what it does to nil and to $\mathsf{cons}(a, \ell)$, assuming its value is already known on ℓ.

For example, length is defined by:

$$\mathsf{length}(\mathsf{nil}) \equiv \mathsf{zero} \qquad \mathsf{length}(\mathsf{cons}(a, \ell)) \equiv \mathsf{succ}(\mathsf{length}(\ell))$$

Likewise, for $f : A \to B$, the map function has type

$$\mathsf{map} : (A \to B) \to \mathsf{List}(A) \to \mathsf{List}(B)$$

and is defined constructor by constructor.

6.2. Where the dependence appears

Once A is fixed, $\mathsf{List}(A)$ is an ordinary inductive type. The list constructor itself is polymorphic, $\mathsf{List} : \mathsf{Type} \to \mathsf{Type}$, while the dependence appears in the eliminator (8), whose target is the varying family $P(\ell)$. A more visibly dependent list-like object records its length in its type. Define vectors by the indexed family

$$\mathsf{Vec}(A) : \mathbb{N} \to \mathsf{Type}$$

with constructors:

$$\mathsf{vnil} : \mathsf{Vec}(A)(\mathsf{zero})$$

$$\mathsf{vcons} : \prod_{n : \mathbb{N}} A \to \mathsf{Vec}(A)(n) \to \mathsf{Vec}(A)(\mathsf{succ}(n))$$

Now the index changes as constructors are applied. The type system can rule out impossible cases. For example,

$$\mathsf{head} : \prod_{n : \mathbb{N}} \mathsf{Vec}(A)(\mathsf{succ}(n)) \to A$$

needs no empty-vector branch: its input type proves that the vector has positive length. Similarly, vector append can state its exact size law in its type:

$$\mathsf{append} : \prod_{m,n : \mathbb{N}} \mathsf{Vec}(A)(m) \to \mathsf{Vec}(A)(n) \to \mathsf{Vec}(A)(m + n)$$

The program and its size invariant are checked simultaneously.

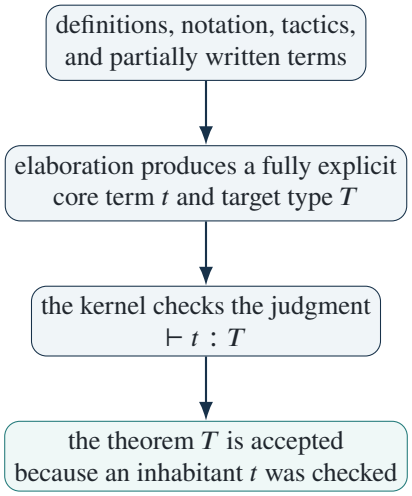
Lists versus vectors

	Type	What the type checker knows
List	$\mathsf{List}(A)$	entries have type A; length is runtime data
Vector	$\mathsf{Vec}(A)(n)$	entries have type A and the length is n

A vector is not a list plus a comment saying what its length should be. Its length is part of the type, so a mismatch becomes a typing error rather than a failed test.

7. What proof verification actually checks

A modern dependent-type proof assistant usually separates convenient user-facing tools from a small trusted core. The workflow is



During type checking, the kernel recursively applies rules such as those for Π-types, existential propositions, equality, and inductive eliminators. It also computes when comparing types:

$$(\lambda x.t)(a) \equiv t[a/x] \qquad \mathsf{Nat.rec}(P, z, s, \mathsf{zero}) \equiv z$$

and similarly for the constructors of lists. Systems such as Rocq and Lean follow the general design in which high-level proof automation produces proof terms that are checked again by the trusted kernel. The kernel is intentionally small and compact, to reduce the chance of bugs.

For the continuity example, the kernel does not decide continuity by numerical testing. It checks that

- the proof accepts every positive ε;
- the existential proof supplies a positive real δ;
- the remaining proof turns arbitrary x and a proof of $|x - 2| < \delta$ into a proof of $|x^2 - 4| < \varepsilon$.

Every use of algebra, order, absolute values, and min must itself be justified by previously verified lemmas or declared axioms.

The limits

A proof assistant checks that a formal term inhabits a formal type relative to the system's rules. It does not guarantee that the formal statement is the statement the author intended. Moreover, it does not prove the theorem for you (even though proof automation tools are rapidly becoming more powerful).

The *de Bruijn factor* is the average ratio between the length of a machine-checkable proof and a human-readable proof of the same statement. Microsoft Research claims that the "formal mathematics revolution" will be complete when Lean reaches a de Bruijn factor of 1. Only time will tell if this happens.

8. Conclusion

If one remembers anything from reading these notes, remember the following three concepts:

Curry–Howard: proving a proposition means constructing a term of the corresponding type.

Dependent types: types may depend on terms; Π-types express dependent functions and, for propositions, universal quantification.

Inductive types: constructors generate data, and dependent eliminators generate programs and proofs by recursion and induction.